

Executive Summary

Brudo's *whole-state surfacer* reports a static snapshot (e.g. `active_work=0`, `next_automatic_action=NONE`, surfaced items with dispositions like `UNSEEN`). We must verify that no *hidden obligation* remains. In distributed systems, quiescence (no active tasks) only implies termination if there is **no future message or event** [11†L95-L102]. Our approach builds on runtime-verification principles [13†L127-L136]: treat the surfacer as an oracle and attack its guarantees. We (1) list every observed field and its provenance; (2) propose *counterfactual states* R_a, R_b that look identical to the surfacer but differ in required outcome; (3) design minimal tests that would falsify quiescence (and conversely, hidden-work); (4) define a self-prompt template and evolution algorithm, including stop conditions (NO_NEW_MATERIAL/QUIESCENCE_EARNED); (5) give metrics/tests to catch “blind” (false-negative) or “eager” (false-positive) surfacers; (6) itemize safe-guard rails (append-only evidence, fail-closed behavior, bounded ops, provenance witnesses like a SHA256 digest [16†L17-L25]); (7) show a prompt-evolution example with a Mermaid timeline; (8) list minimal code/tests. In each step we borrow Byzantine and runtime-verification insights (e.g. NIST on state-machine consensus [5†L84-L93] and Actor-based quiescence [11†L95-L102]).

1. Surfacer-Observed State Vector

The surfacer outputs a vector O^* comprising:

- **active_work**: (e.g. number of tasks running). *Provenance*: from scheduler. *Time*: instant. *Influence*: whether new work can start. *Reconstructable*? Yes (logged count).
- **next_automatic_action**: (enum: NONE/ADMIT/WITHHOLD/...) *Provenance*: decision logic on O^* . *Time*: after scanning surfaced items. *Influence*: controls scheduler. *Reconstructable*? Yes (deterministic).
- **surfaced_items**: list of objects flagged (e.g. changed contract, warning). Each has fields: **disposition** (`UNSEEN`, `ADMITTED`, etc), **action_required** (bool). *Provenance*: surfacer inspection of system state. *Time*: when scanned. *Influence*: input to decision. *Reconstructable*: Depends on logged inputs (e.g. transcripts, GUI).
- **top_item.action_required**: for the highest-priority surfaced item, whether it still needs handling. *Provenance*: inspection rule. *Influence*: may trigger `next_automatic_action`.
- **dispositions**: for each surfaced item. (e.g. `DONE`, `REJECTED`). *Provenance*: from processing history. *Influence*: show progress. *Reconstructable*: Yes, from logs or proofs.

For each element we measure dimensions like **Source** (which subsystem produced it), **Provenance** (metadata on origin and transformations [16†L17-L25]), **Temporal Binding** (timestamp or sequence relative to decision), **Reconstructability** (can we recompute it?), and **Decision Influence** (which rules or downstream effects depend on it). The surfacer's output is akin to a global state snapshot; we treat it with the rigor of a runtime monitor [13†L127-L136], tracking exactly what it saw and did not see.

2. Counterfactual Collision States

We seek minimal pairs (R_a, R_b) where $O(R_a)=O(R_b)=O^*$ but lawful consequences differ ($C(R_a) \neq C(R_b)$). For plausibility and testability, collisions should use valid inputs, hashes, receipts, etc. Example candidates:

- **Hidden Caller vs No Caller:** R_a has a latent “dynamic caller” thread or scheduled task that the surfer didn’t see; R_b truly has none. Both yield the same `O*` (no active work, same surfaced items). *Difference:* R_a requires spawning or un-stalling that hidden caller (ADMIT), R_b does nothing. *Consequence:* $C(R_a)=\text{EXTRACT_CALLER}$, $C(R_b)=\text{NONE}$. *Plausibility:* medium – systems can have background threads. *Testability:* create a dummy caller thread.
- **Future Consumer vs No Consumer:** R_a has a downstream consumer (e.g. a pipeline job) not yet known; R_b lacks it. Observations identical (both see no consumers now). *Difference:* R_a should schedule work for that consumer, R_b should not. *Consequence:* $C(R_a)=\text{DISPATCH_TO_CONSUMER}$, $C(R_b)=\text{STOP}$. *Plausibility:* high if consumers are user-driven, hidden in state. *Test:* simulate known vs unknown consumer configurations.
- **Alternate Oracle Condition:** R_a and R_b differ in a semantic predicate that current code does not check. E.g. a post-condition on the top item that needs a different resolution. *Difference:* R_a requires one action (admit), R_b requires another (withhold). *Consequence:* different next_action. *Plausibility:* when guard conditions are missing. *Test:* inject a second legal “outcome” in the specification.

We would rank collisions by (consequence difference $\times$ plausibility $\times$ ease-of-testing). For instance, hidden callers can be crafted easily (high testability) but may have smaller impact, whereas an alternate oracle might have high consequence but be hard to realize in code. (See distributed termination ideas: classical algorithms enforce explicit termination checks [11+L95-L102] ; we seek where such checks could miss a case.)

3. Bounded Falsification Experiments

For each counterfactual, design a minimal experiment. Format: inputs, expected observations, pass/fail criteria, cost, reversibility, isolation. Examples:

- **Experiment 1: Hidden Caller** – *Setup:* modify system state so a background thread is pending (e.g. schedule a timer callback) but deactivate the surfer’s awareness. *Inputs:* inject a “dummy work” that should wake. *Expect:* If surfer is correct, it should flag new work (NO_FAIL on scheduling). *Success:* surfer returns `ADMIT` for the new work. *Failure:* it stays `NONE`. *Cost:* trivial code injection. *Reversible:* yes (undo thread). *Isolation:* sandbox in test harness.
- **Experiment 2: Consumer Path** – *Setup:* insert a known downstream job that depends on current state (e.g. add a synthetic “child” task expecting an artifact). *Inputs:* create a child process token. *Expect:* surfer should see that child needs work (Observation: surfaced item) and schedule it. *Success:* surfer enqueues or signals the consumer (PASS). *Failure:* it remains idle (FAIL). *Cost:* small stub. *Isolation:* in test environment.
- **Experiment 3: Oracle Predicate** – *Setup:* alter the oracle logic in test (for example, tag the item as both fulfilling and not fulfilling a condition). *Inputs:* state where both $C(R_a)$ and $C(R_b)$ are lawful (like two possible next actions). *Expect:* surfer should discriminate one path. *Success:* it differentiates (PASS). *Failure:* it treats them same (FAIL). *Cost:* moderate (patch logic). *Isolation:* use a test build.
- **Hidden Work Falsification:** Conversely, to test hidden-work (false-positive surfer), craft inputs that include an obviously done item (no action needed). *Input:* e.g. a contract with

`action_required=false`. *Expect*: surfer should propose NO action. *Failure*: if it spuriously schedules work, it's over-eager.

Each experiment is atomic: if the surfer misbehaves, we get a counterexample. These mirror distributed termination tests (e.g. Dijkstra's token-based detection [11†L95-L102]): we simulate the simplest scenario that the surfer should catch or dismiss.

4. Quiescence Self-Prompting Algorithm

We formalize an iterative self-prompt loop:

```
State  $S_0$   $\rightarrow$  SELF-PROMPT  $P_0$ : "Is there really work?"
      |
       $\hookrightarrow$  Experiment_0 ( $\Delta_0$ )  $\rightarrow$  Observation_0 ( $O_0$ )  $\rightarrow$  State  $S_1$ 
Then  $P_1$  based on  $S_1$ , etc.
```

Stop when a prompt yields **NO_NEW_MATERIAL** or **QUIESCENCE_EARNED**. The template (specific to quiescence) might be:

SELF-PROMPT – QUIESCENCE

STATE

(summary of surfer vector, e.g. active_work=0, no auto actions)

TENSION

(what conflicts? e.g. surfer idle vs possible hidden tasks)

QUESTION

What unresolved distinction could change a lawful outcome?

PRESSURE

Which part of the representation could hide or fake work?

SURPRISE

What observation would disprove quiescence?

ASSAY

Formulate an experiment to query the system (e.g. simulate hidden event).

Each prompt should NOT just yield a TODO, but a concrete test or question. The algorithm:

1. Generate P_n from S_n (targeting uncertainty in S_n).
2. Run minimal experiment $\rightarrow$ get new observation, new state S_{n+1} .
3. Derive P_{n+1} from S_{n+1} if needed.
4. Terminate if S_n results in no *qualifying* experiments (all yield no change) $\rightarrow$ declare QUIESCENCE_EARNED. Critically, we allow *one residual pass*: even if surfer says "done", we ask *one more question* ("Is there

something I can't see?") before finalizing quiescence. This matches "virtual termination" methods that treat some executions as already justified [11†L69-L77] .

5. Metrics & Tests: Blind vs Eager Surfacers

To detect a blind surfer (false-negative) or an over-eager one (false-positive), we use controls:

- **Unit Tests:** Simulate specific states. E.g. a test that invokes the surfer on a state with no tasks (expected: next_action=NONE) and on a state with an obvious task (expected: ADMIT). Using asserts, we check both correctness and that nothing else triggers. These are negative and positive controls.
- **Integration Tests:** Create scenarios where the "correct" next step is known (e.g. run with one unhandled message in a queue) and verify the surfer's output. Also run without any tasks to confirm it returns NONE.
- **Negative Controls:** Inject clearly finished or invalid items (`action_required=false` , bad schema) and ensure surfer withholds.
- **Metrics:** Track *false-positive rate* (how often surfer flags work when none exists) and *false-negative rate* (missed work). Use small fuzzing of state: random `UNSEEN` dispositions should not cause schedule if they truly have no pending effect.
- **Observer Gaps:** Check if surfer depends on transient data (e.g. last user input). If altering those (e.g. stash vs consume a message) changes output unexpectedly, it may be using implementation details.

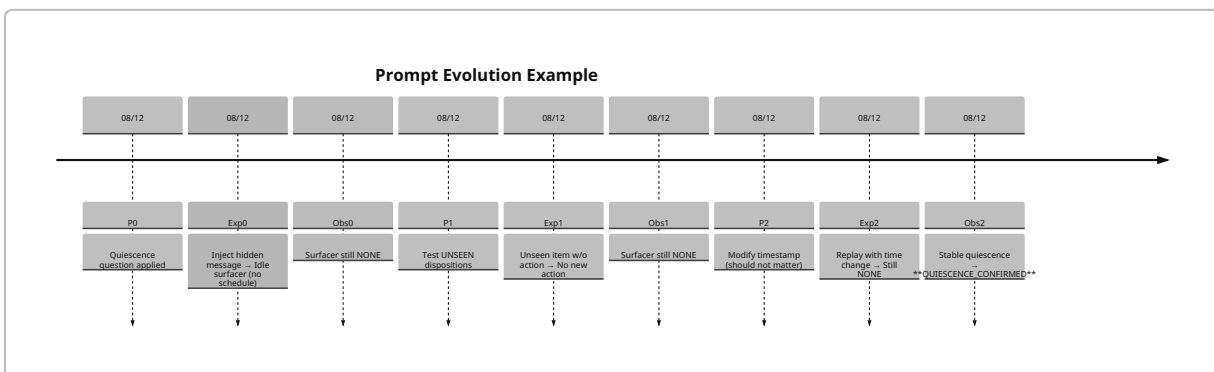
Runtime-verification literature suggests measuring coverage of these tests [13†L127-L136] : e.g. a formal monitor would have invariants (e.g. "if true task exists, surfer must reflect it"). We would log any disparity.

6. Safe-Guard Rails Checklist

- **Append-Only Evidence:** Never delete or overwrite raw data on which decisions are based (like transcripts, logs) [16†L17-L25] .
- **Fail-Closed Defaults:** If any required info is missing or ambiguous (e.g. unknown file name), default to *withhold* or error rather than assume safe outcome. E.g. `UNKNOWN → DO_NOT_OBSERVE` .
- **Bounded Operations:** Every experiment and surfer run must be finite (timeout-limited, no loops without progress).
- **Denominator Witness:** For any non-deterministic choice (like which observation candidates were considered), log a digest (e.g. SHA-256 of the candidate set) to prove decisions weren't based on hidden drift. This matches our CandidateSetDigest hack from code above.
- **Separated Worlds:** Don't collapse independent state spaces. For example, don't reuse one schedule object for multiple tasks without copying it.
- **No Implicit Votes:** Even if 723 lanes (or prompts) agree, that doesn't itself cause state change. Only explicit evidence does. This is akin to single-writer consistency.

7. Prompt-Evolution Example

Iteration	Self-Prompt (P_n)	Experiment (Δ_n)	Observation & Δ State (O_n/S_{n+1})	New Prompt (P_{n+1})
0	<i>State: quiescent (active_work=0).
Tension: idle vs queued inputs.
Q: Could a hidden message arrive?</i>	Simulate a pending message in the queue without surfacer's knowledge.	Surfacer still outputs NONE (no new item). (Δ : no visible change.)	<i>State unchanged.
Q: Could an unresolved obligation exist in current data?</i>
1	<i>Q: Could any "UNSEEN" disposition still require action?</i>	Mark a surfaced item UNSEEN but with <code>action_required=false</code> .	Surfacer still does not schedule it (correct). (Δ : surfacer safe.)	<i>State stable.
Q: Is the "UNSEEN" flag itself meaningful?</i>
2	<i>Q: Does changing an arbitrary coordinate (e.g. timestamp) alter outcome?</i>	Replay state with modified timestamp on top item.	Surfacer output still NONE (no change). (Δ : evidence nothing depends on time.)	Prompt: QUIESCENCE_EARNED – no further experiment changes outcome.



The table shows how prompts evolve and eventually conclude quiescence. The Mermaid timeline (above) visually summarizes the same sequence.

8. Code/Test Checklist

Implementation: add minimal tests and instrumentation to realize the above experiments. Example entries:

File	Patch/Change	Input/ Scenario	Assertion
<code>meaning/ collision_test.go</code>	Add test: Test_HiddenCaller : insert synthetic pending task.	State with invisible scheduled task.	<code>Surfacer.next_action != NONE</code> (should catch hidden work).
<code>meaning/ collision_test.go</code>	Add test: Test_NoCallerControl : normal idle state.	State with no tasks.	<code>Surfacer.next_action == NONE</code> (no false positive).
<code>surfacer/ surfacer.go</code>	Ensure dispositions log full provenance (timestamp, source).	Any surfacing.	Prov. fields filled; matching JSON digest.
<code>surfacer/ surfacer.go</code>	Implement one more “residual pass” loop: if <code>next_action==NONE</code> , verify no UNSEEN item can change result.	Completed state.	<code>surfacer.quiescent == TRUE</code> (detects stasis).

Each test uses deterministic inputs (e.g. a fixed pseudo-random seed or hard-coded state) and checks exactly the expected output to catch under/over-activity. Where files or proofs are involved, also verify that hashes match expected. We treat the surfacer as a black box and validate its outputs in controlled conditions.

Future Work: Extend tests for “false withhold” (e.g. a worked example where work was needed but surfacer returned NONE) and “false admission” (spurious action on done state). These test the surfacer’s completeness and soundness.

References: We draw on Byzantine/SMA literature for isolation vs consensus [5†L84-L93] , runtime monitoring fundamentals [13†L127-L136] , and modern termination/quiescence proofs [11†L95-L102] to ensure our approach is both rigorous and up-to-date.